

TASK 1 - NGƯỜI 3

Motion Primitive + Pick-Place

Tài liệu nội bộ của BA x AILab

Hạng mục	Nội dung
Phiên bản	v1.0 FULL
Đối tượng	Bảo/Vinh+Huy/Trung+Công/Trúc : trong nhóm 4 sinh viên thực hiện Task 1; PM hỗ trợ baseline/config/integration tổng thể
Vai trò	Thiết kế, triển khai, kiểm thử và tối ưu motion primitive pick-place cho 4 workpieces
Mục tiêu cuối	Từ ActionPlan của Người 2, robot gấp từng vật và đặt vào đúng bin bằng chuỗi waypoint an toàn, có retry, timeout, success/failure reason và log đo được
Input chính	object_pose_base, bin_pose_base, grasp_hint, class_id, retry_policy
Output chính	PrimitiveResult, trajectory log, gripper report, pick_place_report, motion failure reason
Module cần tự học	Module 2, 3, 6, 9, 10; Module 4/5 chỉ học mức hiểu input từ perception

Lưu ý: Người 3 không chỉ viết một hàm move_to_pose. Nhiệm vụ là tạo một motion primitive có precondition, waypoint, gripper logic, success check, timeout, retry và failure reason. Một perception tốt vẫn thất bại nếu primitive hạ tay quá nhanh, gấp sai yaw, không confirm grasp hoặc đặt vật ngoài bin.

Mục lục định hướng

- 1. Vai trò của Người 3 trong Task 1
- 2. Người 3 cần tự học module nào
- 3. Input/output contract của Người 3
- 4. Cấu trúc thư mục và artifact cần tạo
- 5. Step-by-step triển khai motion primitive
- 6. Thiết kế waypoint sequence cho Task 1
- 7. Gripper control và grasp confirmation
- 8. Precondition, timeout, retry và failure reason
- 9. Kế hoạch 5 ngày cho Người 3
- 10. Cách phối hợp với Người 1, Người 2, Người 4 và PM
- 11. Debug cookbook cho motion primitive
- 12. Definition of Done, gói nộp và rubric 100 điểm
- 13. Phụ lục code skeleton và schema mẫu
- 14. Tài liệu tham khảo nội bộ và mở

1. Vai trò của Người 3 trong Task 1

Người 3 chịu trách nhiệm biến ActionPlan do Người 2 tạo ra thành hành động robot thật trong mô phỏng. Nói đơn giản: nếu Người 1 giúp robot nhìn thấy vật, Người 2 quyết định vật nào cần xử lý và bin nào là đích, thì Người 3 phải làm cho robot gấp được vật đó và đặt vào đúng bin một cách ổn định.

Câu hỏi Người 3 phải trả lời	Ý nghĩa kỹ thuật
Target pose có nằm trong workspace không?	Nếu không kiểm tra trước, IK/motion sẽ fail hoặc robot di chuyển nguy hiểm.
Nên tiếp cận vật từ hướng nào?	Tạo pre-grasp pose, grasp pose và approach axis phù hợp.
Gripper mở/đóng lúc nào?	Đóng quá sớm/quá muộn đều làm rơi hoặc đẩy vật.
Nâng vật cao bao nhiêu?	Lift không đủ cao có thể va vào bàn hoặc vật khác.
Đặt vật vào vị trí nào trong bin?	Đặt vào tâm bin với margin an toàn giúp giảm wrong bin/drop.
Khi primitive thất bại thì log gì và retry thế nào?	Để Người 4 phân tích lỗi và cả nhóm tối ưu có hệ thống.

Mục tiêu thực dụng: Bản đầu không cần nhanh. Ưu tiên success rate: gấp chậm, đặt chậm, waypoint an toàn, log đầy đủ. Sau khi ổn định mới giảm thời gian.

2. Người 3 cần tự học module nào

Thứ tự	Module	Mức độ	Cần lấy được cho Task 1
1	Module 2 - Isaac Sim, USD/URDF, Walker S2, Baseline	Học khá sâu phần robot/action	Biết robot root, joint/gripper/end-effector path, action interface và cách baseline nhận lệnh.
2	Module 3 - Robotics Math	Học sâu phần pose/quaternion/IK/workspace	Biết đọc pose_base, tạo target pose, hiểu quaternion/yaw, nhận ra target ngoài workspace.
3	Module 6 - Motion Primitives and Dual-Arm Control	Học sâu Task 1	Biết thiết kế primitive có precondition, waypoint, success check, retry, failure reason.
4	Module 9 - Task-by-Task Solution Design	Học phần Task 1	Hiểu FSM và primitive nằm ở state EXECUTE PICK PLACE.
5	Module 10 - Evaluation/Logging/Ablation	Học đủ dùng	Biết log drop, collision, IK fail, retry, duration.
6	Module 4/5	Đọc lướt input contract	Hiểu pose_base và grasp_hint từ Người 1; không cần tự làm perception.

3. Input/output contract của Người 3

Người 3 không nên đọc trực tiếp ảnh hoặc tự chọn vật. Người 3 nhận ActionPlan từ Người 2 và trả PrimitiveResult cho Người 4/runner. Làm đúng contract giúp nhóm không bị rối khi tích hợp.

3.1. Input: Planner → Motion

```
{
  "primitive": "pick_place",
  "object_id": "obj_001",
  "class_id": "part_A",
  "object_pose_base": {
    "position_m": [0.42, -0.15, 0.82],
    "quaternion_xyzw": [0.0, 0.0, 0.707, 0.707]
  },
  "target_bin": "bin_A",
  "bin_pose_base": {
    "position_m": [0.55, 0.20, 0.80],
    "quaternion_xyzw": [0.0, 0.0, 0.0, 1.0]
  },
  "grasp_hint": {
    "yaw_rad": 1.57,
    "grasp_width_m": 0.045,
    "approach_axis": "z_down"
  },
  "retry_policy": "retry_once_slow"
}
```

3.2. Output: Motion → Evaluation/Runner

```
{
  "object_id": "obj_001",
  "primitive": "pick_place",
  "primitive_success": true,
  "duration_s": 6.4,
  "retry_count": 0,
  "failure_reason": null,
  "waypoints_executed": ["pre_grasp", "grasp", "close", "lift", "pre_place", "place", "release",
"retreat"],
  "artifacts": {
    "trajectory_csv": "logs/task1/traj_obj_001.csv",
    "gripper_report": "logs/task1/gripper_obj_001.json"
  }
}
```

Trường	Người 3 cần dùng như thế nào
object_pose_base.position_m	Tạo grasp pose, pre_grasp pose và lift pose.
object_pose_base.quaternion_xyzw	Định hướng vật; bản MVP có thể chỉ dùng yaw từ grasp_hint.
bin_pose_base	Tạo pre_place và place pose ở tâm bin hoặc vùng an toàn trong bin.
grasp_hint.yaw_rad	Xoay gripper theo hướng gấp; nếu chưa ổn thì thử yaw và yaw + $\pi/2$.
grasp_hint.grasp_width_m	Đặt độ mở/đóng gripper phù hợp với kích thước vật.
retry_policy	Quyết định retry chậm hơn, cao hơn hoặc bỏ qua object.

4. Cấu trúc thư mục và artifact cần tạo

```
src/task1/
  motion.py
  motion_utils.py
  gripper_utils.py
  trajectory_utils.py
  motion_debug.py

configs/
  task1_motion.yaml
  workspace_limits.yaml

lab_outputs/motion/
  motion_interface_note.md
  primitive_spec_task1.yaml
  waypoints_task1.csv
  gripper_test_report.csv
  motion_dryrun_report.md
  trajectory_metrics.csv
  primitive_result_schema.json
  pick_place_report.csv
  motion_failure_cases.jsonl

logs/task1/
  traj_obj_001.csv
  gripper_obj_001.json
  primitive_results.jsonl
```

5. Step-by-step triển khai motion primitive

Step 1. Nhận thông tin robot/action interface từ PM và baseline

Trước khi viết motion, Người 3 cần biết baseline đang điều khiển robot bằng joint target, end-effector target, delta pose hay policy action. Không được giả định action interface.

Cần biết	Câu hỏi cần trả lời	Output
Robot root/articulation	Walker S2 articulation root ở path nào?	motion_interface_note.md
End-effector	Tên/path end-effector hoặc gripper frame là gì?	motion_interface_note.md
Gripper interface	Mở/đóng bằng joint, command, surface gripper hay API nào?	gripper_test_report.csv
Action format	Baseline nhận lệnh dạng joint positions, EE pose hay delta action?	motion_interface_note.md
Control frequency	Mỗi step gửi action ở tần số nào?	trajectory_metrics.csv
Workspace limits	Vùng tay với tới an toàn là gì?	workspace_limits.yaml

```
# motion_interface_note.md - mẫu nội dung
- Robot root: ...
- Left/right arm used for Task 1: ...
- End-effector frame: ...
- Gripper command: open=..., close=...
- Action API function/class: ...
- Control frequency: ... Hz
- Workspace limits: x=[...], y=[...], z=[...]
- Known constraints: ...
```

Step 2. Viết primitive spec trước khi code

Primitive spec giúp cả nhóm hiểu motion sẽ làm gì. Người 3 cần viết spec trước, để Người 2 biết phải truyền gì và Người 4 biết cần log gì.

```
name: pick_place_workpiece_task1
owner: person3_motion
inputs:
  object_pose_base: SE3
  class_id: part_A_or_B
  bin_pose_base: SE3
  grasp_hint:
    yaw_rad: float
    grasp_width_m: float
    approach_axis: z_down
preconditions:
  - confidence >= 0.70
  - object_pose_base inside reachable_workspace
  - bin_pose_base valid
  - gripper is open
waypoints:
  - pre_grasp
  - grasp
  - close_gripper
  - lift
  - pre_place
  - place
  - open_gripper
  - retreat
success_checks:
  - primitive finished without IK_FAIL/TIMEOUT/COLLISION
  - gripper closed before lift
  - no drop detected during transfer if signal available
recovery:
  - retry once with higher pre_grasp and slower descend
  - if second fail, return failure_reason and let planner choose next action
```

Step 3. Tạo hàm kiểm tra precondition

Precondition giúp phát hiện lỗi sớm trước khi robot chạy. Nếu object pose không hợp lệ hoặc target ngoài workspace, primitive phải fail có lý do rõ ràng thay vì gửi lệnh nguy hiểm.

```
def check_preconditions(action_plan, workspace_limits):
    obj = action_plan["object_pose_base"]["position_m"]
    bin_pos = action_plan["bin_pose_base"]["position_m"]
    grasp_hint = action_plan.get("grasp_hint", {})

    for name, p in [("object", obj), ("bin", bin_pos)]:
        if len(p) != 3:
            return False, f"BAD_{name.upper()}_POSE"
        x, y, z = p
        if not (workspace_limits["x"][0] <= x <= workspace_limits["x"][1]):
            return False, f"{name.upper()}_OUT_OF_WORKSPACE_X"
        if not (workspace_limits["y"][0] <= y <= workspace_limits["y"][1]):
            return False, f"{name.upper()}_OUT_OF_WORKSPACE_Y"
        if not (workspace_limits["z"][0] <= z <= workspace_limits["z"][1]):
            return False, f"{name.upper()}_OUT_OF_WORKSPACE_Z"

    if grasp_hint.get("grasp_width_m", 0) <= 0:
        return False, "INVALID_GRASP_WIDTH"

    return True, None
```

Step 4. Tạo waypoint generator

Waypoint generator là trái tim của pick-place. Từ object pose và bin pose, tạo các pose trung gian an toàn. Bản MVP có thể dùng orientation đơn giản: giữ gripper hướng xuống, yaw theo grasp_hint.

Waypoint	Mục đích	Cách tạo gợi ý
pre_grasp	Đến phía trên vật, tránh va chạm	object position + [0,0,+0.10]
grasp	Hạ xuống vị trí gấp	object position + grasp z offset
lift	Nâng vật lên khỏi bàn	object position + [0,0,+0.12]
pre_place	Đến phía trên bin	bin position + [0,0,+0.12]
place	Hạ vào bin	bin position + place z offset, ở tâm bin
retreat	Rút tay lên sau khi thả	bin position + [0,0,+0.15]

```
import math

def make_pose(position, yaw_rad, z_down=True):
    # Placeholder. Trong baseline thật cần dùng convention quaternion của hệ thống.
    # MVP: tạo quaternion từ yaw quanh trục z và gripper hướng xuống nếu controller hỗ trợ.
    return {
        "position_m": position,
        "yaw_rad": yaw_rad,
        "approach_axis": "z_down" if z_down else "custom"
    }

def generate_pick_place_waypoints(action_plan, params):
    obj = action_plan["object_pose_base"]["position_m"]
    bin_pos = action_plan["bin_pose_base"]["position_m"]
    yaw = action_plan.get("grasp_hint", {}).get("yaw_rad", 0.0)

    h_pre = params.get("pre_grasp_height_m", 0.10)
    h_lift = params.get("lift_height_m", 0.12)
    h_place = params.get("pre_place_height_m", 0.12)
    grasp_z_offset = params.get("grasp_z_offset_m", 0.00)
    place_z_offset = params.get("place_z_offset_m", 0.02)

    waypoints = [
        ("pre_grasp", make_pose([obj[0], obj[1], obj[2] + h_pre], yaw)),
        ("grasp", make_pose([obj[0], obj[1], obj[2] + grasp_z_offset], yaw)),
        ("lift", make_pose([obj[0], obj[1], obj[2] + h_lift], yaw)),
        ("pre_place", make_pose([bin_pos[0], bin_pos[1], bin_pos[2] + h_place], yaw)),
        ("place", make_pose([bin_pos[0], bin_pos[1], bin_pos[2] + place_z_offset], yaw)),
        ("retreat", make_pose([bin_pos[0], bin_pos[1], bin_pos[2] + h_place + 0.03], yaw)),
    ]
    return waypoints
```

Step 5. Ghi waypoints ra CSV để debug

Mỗi lần primitive chạy, phải ghi waypoint ra CSV. Nếu robot gặp lệch, CSV giúp kiểm tra target pose trước khi nghi IK/perception.

```
import csv

def write_waypoints_csv(path, object_id, waypoints):
    with open(path, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["object_id", "waypoint", "x", "y", "z", "yaw_rad"])
        for name, pose in waypoints:
            x, y, z = pose["position_m"]
            yaw = pose.get("yaw_rad", 0.0)
            writer.writerow([object_id, name, x, y, z, yaw])
```

6. Thiết kế waypoint sequence cho Task 1

Bước	Action	Lưu ý triển khai	Failure cần log
1	move_to(pre_grasp)	Di chuyển cao hơn vật 8-12 cm; tốc độ vừa phải.	IK_FAIL_PRE_GRASP, OUT_OF_WORKSPACE
2	open_gripper()	Bảo đảm gripper đang mở trước khi hạ.	GRIPPER_OPEN_FAIL
3	move_to(grasp)	Hạ chậm; tránh hạ giật cục.	IK_FAIL_GRASP, COLLISION_ON_DESCEND
4	close_gripper()	Đóng theo grasp_width; wait ngắn để ổn định.	GRIPPER_CLOSE_FAIL
5	confirm_grasp()	Nếu có sensor/contact hoặc object vẫn nằm giữa gripper thì pass.	GRASP_NOT_CONFIRMED
6	move_to(lift)	Nâng thẳng lên đủ cao trước khi đi ngang.	DROP_DURING_LIFT
7	move_to(pre_place)	Di chuyển trên cao đến bin.	IK_FAIL_PRE_PLACE, COLLISION_TRANSFER
8	move_to(place)	Hạ chậm vào tâm bin, có margin.	IK_FAIL_PLACE, WRONG_PLACE_HEIGHT
9	open_gripper()	Thả vật; wait ngắn để vật ổn định.	RELEASE_FAIL
10	move_to(retreat)	Rút tay lên tránh kéo vật ra khỏi bin.	IK_FAIL_RETREAT

Nguyên tắc waypoint: Bản MVP nên ưu tiên đường đi đơn giản: đi thẳng xuống để gấp, nâng lên cao, đi ngang khi đã nâng đủ, hạ xuống bin chậm, thả, rồi rút lên. Không tối ưu thời gian ở bản đầu.

7. Gripper control và grasp confirmation

Gripper là nguồn lỗi lớn trong Task 1. Người 3 cần tách riêng test gripper trước khi nối pick-place đầy đủ.

Test	Cách làm	Output
Open/close command	Gửi open, wait, gửi close, wait, ghi trạng thái nếu có.	gripper_test_report.csv
Width sanity	Thử nhiều grasp width: nhỏ, vừa, lớn.	gripper_width_sweep.csv
Close timing	Thử wait sau close: 0.1s, 0.3s, 0.5s.	gripper_timing_report.csv
Lift confirmation	Sau close, lift nhẹ 2-3 cm, kiểm tra object có đi theo không nếu có sensor/vision.	grasp_confirmation_report.md

```
def open_gripper(robot, params):
    # TODO: thay bằng API baseline thật
    robot.command_gripper(width=params.get("open_width_m", 0.08))
    wait_steps(params.get("open_wait_steps", 10))

def close_gripper(robot, grasp_width_m, params):
    # Nên giới hạn grasp_width trong khoảng an toàn
    min_w = params.get("min_grasp_width_m", 0.01)
    max_w = params.get("max_grasp_width_m", 0.08)
    width = max(min_w, min(max_w, grasp_width_m))
    robot.command_gripper(width=width)
    wait_steps(params.get("close_wait_steps", 15))

def confirm_grasp(robot, observation=None):
    # MVP: nếu chưa có sensor, trả True sau close và lift thử.
    # Bản tốt hơn: dùng gripper width/contact hoặc vision để xác nhận object đã rời bàn.
    return True, None
```

8. Precondition, timeout, retry và failure reason

8.1. Failure reason chuẩn cho Người 3

Failure reason	Khi nào dùng	Cách sửa ưu tiên
BAD_ACTION_PLAN	Thiếu object_pose/bin_pose/grasp_hint.	Yêu cầu Người 2 kiểm tra schema.
OBJECT_OUT_OF_WORKSPACE	Object pose ngoài vùng với tới.	Chọn object khác hoặc kiểm tra transform.
BIN_OUT_OF_WORKSPACE	Bin pose ngoài vùng với tới.	Kiểm tra bin config.
IK_FAIL_PRE_GRASP	Không đến được pre_grasp.	Tăng height hoặc kiểm tra pose.
IK_FAIL_GRASP	Không đến được grasp pose.	Điều chỉnh z offset/yaw.
GRIPPER_OPEN_FAIL	Không mở được gripper.	Kiểm tra API gripper.
GRIPPER_CLOSE_FAIL	Không đóng được gripper.	Kiểm tra grasp_width và command.
GRASP_NOT_CONFIRMED	Đóng gripper nhưng không giữ vật.	Chỉnh yaw, grasp point, close wait.
DROP_DURING_LIFT	Vật rơi sau khi nâng.	Giảm tốc lift, chỉnh grasp_width/yaw.
COLLISION_TRANSFER	Va chạm khi di chuyển sang bin.	Tăng lift/pre_place height.
PLACE_FAIL	Không thả hoặc đặt sai vị trí.	Chỉnh place z, bin center, wait sau release.
TIMEOUT	Primitive chạy quá lâu.	Giảm waypoint thừa sau khi ổn định.

8.2. Retry policy tối thiểu

Retry không nên vô hạn. Task 1 nên retry một lần với thông số an toàn hơn. Nếu vẫn fail, trả failure để planner quyết định chọn vật khác hoặc fail episode.

Lỗi	Retry đề xuất
IK_FAIL_GRASP	Tăng pre_grasp height +0.03 m, thử yaw + $\pi/2$ nếu vật dài.
GRASP_NOT_CONFIRMED	Đóng chậm hơn, tăng close_wait_steps, thử grasp_width nhỏ hơn.
DROP_DURING_LIFT	Giảm tốc nâng, nâng thẳng đứng lâu hơn trước khi transfer.
COLLISION_TRANSFER	Tăng lift/pre_place height, đi theo đường vòng cao hơn.
PLACE_FAIL	Đặt vào tâm bin, tăng margin, hạ chậm hơn.

```
def apply_retry_adjustment(params, failure_reason):
    new_params = dict(params)
    if failure_reason in ["IK_FAIL_GRASP", "COLLISION_ON_DESCEND"]:
        new_params["pre_grasp_height_m"] = params.get("pre_grasp_height_m", 0.10) + 0.03
        new_params["descend_speed_scale"] = 0.5
    elif failure_reason in ["GRASP_NOT_CONFIRMED", "DROP_DURING_LIFT"]:
        new_params["close_wait_steps"] = params.get("close_wait_steps", 15) + 10
        new_params["lift_speed_scale"] = 0.5
    elif failure_reason == "COLLISION_TRANSFER":
        new_params["lift_height_m"] = params.get("lift_height_m", 0.12) + 0.05
    return new_params
```

9. Kế hoạch 5 ngày cho Người 3

Ngày	Mục tiêu	Việc cụ thể	Output bắt buộc
1	Hiểu action/gripper interface	Đọc Module 2/6; hỏi PM action API; viết motion_interface_note.	motion_interface_note.md
2	Primitive spec + waypoint generator	Viết primitive_spec, generate waypoint từ pose giả lập, ghi CSV.	primitive_spec_task1.yaml, waypoints_task1.csv
3	Gripper và motion dry-run	Test open/close, move_to pose mẫu, log result.	gripper_test_report.csv, motion_dryrun_report.md
4	Nối với Planner ActionPlan	Nhận JSON từ Người 2, chạy pick_place với object/bin pose thật hoặc mock.	primitive_results.jsonl, trajectory_metrics.csv
5	Tối ưu failure top 3	Dựa vào log của Người 4: sửa drop, IK fail, collision.	pick_place_report.csv, motion_failure_cases.jsonl

10. Cách phối hợp với Người 1, Người 2, Người 4 và PM

Phối hợp với	Người 3 cần nhận	Người 3 cần trả lại
PM	Robot/gripper/action API, scene/config path, cách chạy baseline.	Danh sách lỗi baseline/action API nếu có.
Người 1 - Perception	grasp_hint, yaw, grasp_width, object pose confidence.	Phản hồi yaw/grasp_width có dùng được không.
Người 2 - Planner/FSM	ActionPlan đúng schema, retry_policy, bin pose.	PrimitiveResult và failure_reason để FSM quyết định.
Người 4 - Evaluation	Log schema và failure taxonomy.	trajectory CSV, duration, drop/collision/IK fail, retry_count.

Quy tắc trao đổi: Nếu motion fail, không kết luận ngay là do motion. Cần kiểm tra theo thứ tự: ActionPlan có đúng không, pose_base có hợp lý không, waypoint có nằm trong workspace không, rồi mới sửa controller/IK/gripper.

11. Debug cookbook cho motion primitive

Lỗi	Triệu chứng	Cách kiểm tra	Cách sửa ưu tiên
Tay gấp lệch vài cm	Gripper xuống gần vật nhưng không trúng	So sánh overlay pose, waypoint CSV, object pose	Kiểm tra transform với Người 1/2; thêm offset calibration nhỏ nếu cần.
IK fail ở pre_grasp	Robot không đi đến waypoint đầu	Log target x/y/z/yaw và workspace	Tăng height, chọn pose ít sát giới hạn, kiểm tra arm dùng đúng bên.
IK fail ở grasp	Đến trên vật nhưng không hạ được	So sánh grasp z với mặt bàn/object height	Tăng grasp_z_offset, hạ chậm hơn, tránh pose quá thấp.
Gripper đẩy vật đi	Khi hạ/close, vật bị đẩy lệch	Xem video/viewport và log yaw	Giảm descend speed, chỉnh yaw, gấp ở tâm ổn định hơn.
Không giữ được vật	Lift lên nhưng vật nằm lại/roi	Kiểm tra close_wait, grasp_width, gripper report	Tăng wait sau close, điều chỉnh grasp_width, thử yaw+pi/2.
Roi khi transfer	Nâng được nhưng rơi khi đi ngang	Log velocity/waypoint duration	Di chuyển chậm hơn, nâng cao hơn, giữ gripper chặt hơn.
Va vào bàn/vật khác	Collision khi đi ngang hoặc hạ	Xem trajectory CSV và video	Tăng lift height, đi qua pre_place cao hơn.
Đặt ngoài bin	Vật rơi sát mép hoặc nằm vùng	Kiểm tra bin_pose và place pose	Đặt vào tâm bin, thêm margin, giảm release height.
Timeout	Primitive chạy quá lâu	duration từng waypoint	Chỉ tối ưu sau khi success ổn; giảm waypoint/wait thừa.

12. Definition of Done, gói nộp và rubric 100 điểm

12.1. Definition of Done cho Người 3

- ☐ Có motion_interface_note.md xác định được action/gripper interface.
- ☐ Có primitive_spec_task1.yaml với precondition, waypoint, success check, retry.
- ☐ Có waypoint generator tạo được pre_grasp, grasp, lift, pre_place, place, retreat.
- ☐ Có waypoints_task1.csv cho ít nhất một object/action plan.
- ☐ Có gripper_test_report.csv chứng minh open/close hoạt động hoặc ghi rõ block.
- ☐ Có motion_dryrun_report.md với pose giả lập hoặc action plan mock.
- ☐ pick_place() trả PrimitiveResult đúng schema.
- ☐ Có failure_reason rõ cho IK fail, gripper fail, drop, collision, timeout.
- ☐ Có retry một lần với tham số an toàn hơn.
- ☐ Người 4 đọc được primitive_results.jsonl để tính metric.

12.2. Gói nộ cá nhân

```
person3_motion_pack/  
  README_MOTION.md  
  src/  
    motion.py  
    motion_utils.py  
    gripper_utils.py  
    trajectory_utils.py  
    motion_debug.py  
  configs/  
    task1_motion.yaml  
    workspace_limits.yaml  
  outputs/  
    motion_interface_note.md  
    primitive_spec_task1.yaml  
    waypoints_task1.csv  
    gripper_test_report.csv  
    motion_dryrun_report.md  
    trajectory_metrics.csv  
    primitive_result_schema.json  
    pick_place_report.csv  
    motion_failure_cases.jsonl  
  logs/  
    primitive_results.jsonl  
    traj_obj_001.csv
```

12.3. Rubric 100 điểm cho Người 3

Hạng mục	Điểm	Tiêu chí đạt
Hiểu action/gripper interface	15	Có note rõ API, end-effector, gripper, workspace, control frequency.
Primitive spec	15	Spec có input, precondition, waypoint, success check, retry, failure reason.
Waypoint generator	15	Tạo đủ pre_grasp/grasp/lift/pre_place/place/retreat và ghi CSV.
Gripper control	15	Có open/close test, grasp_width, timing, report rõ.
Pick-place implementation	20	Chạy được dry-run hoặc simulation; trả PrimitiveResult đúng schema.
Failure handling/retry	10	Có timeout, failure_reason, retry_once_slow.
Logging/debug artifact	10	Có trajectory_metrics, motion_failure_cases, report dễ đọc.

13. Phụ lục code skeleton và schema mẫu

13.1. motion.py skeleton

```
import time  
  
class MotionPrimitiveRunner:  
    def __init__(self, robot, params, workspace_limits):  
        self.robot = robot  
        self.params = params  
        self.workspace_limits = workspace_limits  
  
    def pick_place(self, action_plan, attempt=0):  
        t0 = time.time()
```

```

object_id = action_plan.get("object_id", "unknown")
executed = []

ok, reason = check_preconditions(action_plan, self.workspace_limits)
if not ok:
    return self._result(object_id, False, time.time() - t0, attempt, reason, executed)

params = self.params
waypoints = generate_pick_place_waypoints(action_plan, params)

try:
    # 1. pre_grasp
    self.move_to_named_waypoint(waypoints, "pre_grasp")
    executed.append("pre_grasp")

    # 2. open
    open_gripper(self.robot, params)
    executed.append("open_gripper")

    # 3. grasp
    self.move_to_named_waypoint(waypoints, "grasp", slow=True)
    executed.append("grasp")

    # 4. close
    grasp_width = action_plan.get("grasp_hint", {}).get("grasp_width_m", 0.045)
    close_gripper(self.robot, grasp_width, params)
    executed.append("close_gripper")

    # 5. confirm
    grasp_ok, grasp_reason = confirm_grasp(self.robot)
    if not grasp_ok:
        return self._maybe_retry(action_plan, attempt, "GRASP_NOT_CONFIRMED", executed, t0)
    executed.append("confirm_grasp")

    # 6. lift
    self.move_to_named_waypoint(waypoints, "lift")
    executed.append("lift")

    # 7. pre_place
    self.move_to_named_waypoint(waypoints, "pre_place")
    executed.append("pre_place")

    # 8. place
    self.move_to_named_waypoint(waypoints, "place", slow=True)
    executed.append("place")

    # 9. release
    open_gripper(self.robot, params)
    executed.append("release")

    # 10. retreat
    self.move_to_named_waypoint(waypoints, "retreat")
    executed.append("retreat")

    return self._result(object_id, True, time.time() - t0, attempt, None, executed)

except IKError as e:
    return self._maybe_retry(action_plan, attempt, "IK_FAIL", executed, t0)
except TimeoutError:
    return self._result(object_id, False, time.time() - t0, attempt, "TIMEOUT", executed)
except Exception as e:

```

```

        return self._result(object_id, False, time.time() - t0, attempt, f"RUNTIME_ERROR:{e}",
executed)

    def move_to_named_waypoint(self, waypoints, name, slow=False):
        pose = dict(waypoints)[name]
        # TODO: thay bằng API move_to_pose thật của baseline
        self.robot.move_to_pose(pose, slow=slow)

    def _maybe_retry(self, action_plan, attempt, reason, executed, t0):
        if attempt < 1:
            self.params = apply_retry_adjustment(self.params, reason)
            return self.pick_place(action_plan, attempt=attempt + 1)
        return self._result(action_plan.get("object_id", "unknown"), False, time.time() - t0, attempt,
reason, executed)

    def _result(self, object_id, success, duration_s, retry_count, failure_reason, executed):
        return {
            "object_id": object_id,
            "primitive": "pick_place",
            "primitive_success": success,
            "duration_s": duration_s,
            "retry_count": retry_count,
            "failure_reason": failure_reason,
            "waypoints_executed": executed
        }

```

13.2. task1_motion.yaml mẫu

```

pre_grasp_height_m: 0.10
lift_height_m: 0.12
pre_place_height_m: 0.12
gripper:
  open_width_m: 0.08
  min_grasp_width_m: 0.01
  max_grasp_width_m: 0.08
  open_wait_steps: 10
  close_wait_steps: 15
speeds:
  approach_speed_scale: 0.8
  descend_speed_scale: 0.5
  lift_speed_scale: 0.7
  transfer_speed_scale: 0.8
retry:
  max_retry: 1
  retry_pre_grasp_height_add_m: 0.03
  retry_slowdown_scale: 0.5

```

14. Tài liệu tham khảo nội bộ và mở

Tài liệu	Dùng để làm gì
HRC2026 Task 1 Solo Guide	Yêu cầu Task 1, pipeline, pick-place sequence, metrics và checklist end-to-end.
HRC2026 Task 1 Team4 Guide	Phân vai 4 người và interface giữa perception, planner, motion, evaluation.
Module 2 - Isaac Sim, USD/URDF, Walker S2 và Baseline Architecture	Đọc robot/action interface, end-effector, gripper, baseline config.
Module 3 - Robotics Math: SE(3), Quaternion, FK/IK	Pose, transform, quaternion, workspace và IK intuition.
Module 6 - Motion Primitives and Dual-Arm Control	Thiết kế primitive có precondition, waypoint, success check, retry.
Module 9 - Task-by-Task Solution Design	FSM Task 1, task contract và hybrid solution design.
Module 10 - Evaluation, Logging, Ablation	Metric, failure taxonomy, episode/step logs.

NVIDIA Isaac Sim 5.1 Documentation	Robot simulation, articulation, controllers, gripper/camera/physics debugging.
Modern Robotics	Nền tảng pose, FK/IK, trajectory và manipulation.

Kết luận: Người 3 hoàn thành tốt khi tạo được motion primitive chạy chậm nhưng chắc, có waypoint rõ, gripper test, retry, failure reason và log đầy đủ. Khi Task 1 chưa ổn định, đừng tối ưu tốc độ; hãy sửa các lỗi lớn nhất theo log: IK fail, grasp fail, drop, collision, wrong place.